

Session 10

System Integration & Debugging



Find out why it doesn't work — then prove that it does.

Part	Topic	Time
01	Debugging like a detective	7 min
02	Serial prints: make the invisible visible	9 min
03	Practice: a full system test of both projects	9 min
04	AI integration: edge cases and robustness	5 min

Builds on everything in M3 — especially the –1 fail-safe from Session 8 and the joystick edge from Session 5.

Six words you will hear today.

Word	Plain meaning
Bug	A mistake that makes a program do something you didn't mean.
Debugging	Finding the bug and fixing it.
Symptom	What you see go wrong — “the pump never starts”. Not the cause.
Hypothesis	Your best guess at the cause, which a test can prove or disprove.
Edge case	An unusual situation at the limits: a sensor unplugged, a button held, the end of the strip.
Test plan	A table of things to try, what should happen, and what did happen.

Debug like a doctor, not a guesser.

A patient says “my head hurts” — the symptom. The doctor forms a hypothesis, runs a test, and only then treats.

Never change code at random. Every change is a test of one idea.

Five steps, every time.

Step	Ask yourself
1. Symptom	What exactly goes wrong? When? Every time?
2. Clue	What do the numbers inside the program say? (Serial prints.)
3. Hypothesis	Which line could cause this?
4. Test	Change one thing, run it again. Did the symptom change?
5. Fix and retest	Fix it, then rerun every test that used to pass.

Add one module at a time.

When the Smart Garden “doesn’t work”, which part is broken — the probe, the conversion, the thresholds?

Add one module, test it, then add the next. If it breaks, the bug is in what you just added.

Make the invisible visible.

The board can't show you its variables — unless you print them, with a label.

Next: the Smart Garden with one small change and a bug. The soil is bone dry, but the pump never starts.

Find the bug with debug prints.

```
1 void loop() {  
2   float raw = incipe.getSoilMoisture();  
3   float wet = wetPercent(raw);  
4   if (raw = -1) wet = 100;           // meant: no reading → act as if wet  
5   Serial.print("[debug] raw=");  
6   Serial.print(raw);  
7   Serial.print(" wet=");  
8   Serial.println(wet);  
9   if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
10  if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }  
11  delay(2000);  
12 }
```

LINE 2 · STEP 1 OF 8

The probe is in dry soil. The sensor gives 85.

VARIABLES

raw = 85
watering = false

SERIAL MONITOR

Find the bug with debug prints.

```
1 void loop() {  
2   float raw = incipe.getSoilMoisture();  
3   float wet = wetPercent(raw);  
4   if (raw = -1) wet = 100;           // meant: no reading → act as if wet  
5   Serial.print("[debug] raw=");  
6   Serial.print(raw);  
7   Serial.print(" wet=");  
8   Serial.println(wet);  
9   if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
10  if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }  
11  delay(2000);  
12 }
```

LINE 3 · STEP 2 OF 8

85 converts to 25 % wet — dry, as expected.

VARIABLES

```
raw = 85  
watering = false  
wet = 25
```

SERIAL MONITOR

Find the bug with debug prints.

```
1 void loop() {  
2   float raw = incipe.getSoilMoisture();  
3   float wet = wetPercent(raw);  
4   if (raw = -1) wet = 100;           // meant: no reading → act as if wet  
5   Serial.print("[debug] raw=");  
6   Serial.print(raw);  
7   Serial.print(" wet=");  
8   Serial.println(wet);  
9   if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
10  if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }  
11  delay(2000);  
12 }
```

LINE 4 · STEP 3 OF 8

Look closely: = not ==. This line puts -1 into raw.

VARIABLES

```
raw = -1  
watering = false  
wet = 25
```

SERIAL MONITOR



Find the bug with debug prints.

```
1 void loop() {  
2   float raw = incipe.getSoilMoisture();  
3   float wet = wetPercent(raw);  
4   if (raw == -1) wet = 100;           // meant: no reading → act as if wet  
5   Serial.print("[debug] raw=");  
6   Serial.print(raw);  
7   Serial.print(" wet=");  
8   Serial.println(wet);  
9   if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
10  if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }  
11  delay(2000);  
12 }
```

LINE 4 · STEP 4 OF 8

The if sees -1, which isn't 0, so it counts as true: wet becomes 100.

VARIABLES

```
raw = -1  
watering = false  
wet = 100
```

SERIAL MONITOR

Find the bug with debug prints.

```
1 void loop() {  
2   float raw = incipe.getSoilMoisture();  
3   float wet = wetPercent(raw);  
4   if (raw == -1) wet = 100;           // meant: no reading → act as if wet  
5   Serial.print("[debug] raw=");  
6   Serial.print(raw);  
7   Serial.print(" wet=");  
8   Serial.println(wet);  
9   if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
10  if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }  
11  delay(2000);  
12 }
```

LINE 6 · STEP 5 OF 8

The debug print shows what the program now believes.

VARIABLES

```
raw = -1  
watering = false  
wet = 100
```

SERIAL MONITOR

```
[debug] raw=-1.00
```

Find the bug with debug prints.

```
1 void loop() {  
2   float raw = incipe.getSoilMoisture();  
3   float wet = wetPercent(raw);  
4   if (raw == -1) wet = 100;           // meant: no reading → act as if wet  
5   Serial.print("[debug] raw=");  
6   Serial.print(raw);  
7   Serial.print(" wet=");  
8   Serial.println(wet);  
9   if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
10  if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }  
11  delay(2000);  
12 }
```

LINE 8 · STEP 6 OF 8

...and the soil looks soaking wet.

VARIABLES

```
raw = -1  
watering = false  
wet = 100
```

SERIAL MONITOR

```
[debug]  
raw=-1.00 wet=100.00
```

Find the bug with debug prints.

```
1 void loop() {  
2   float raw = incipe.getSoilMoisture();  
3   float wet = wetPercent(raw);  
4   if (raw = -1) wet = 100;           // meant: no reading → act as if wet  
5   Serial.print("[debug] raw=");  
6   Serial.print(raw);  
7   Serial.print(" wet=");  
8   Serial.println(wet);  
9   if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
10  if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }  
11  delay(2000);  
12 }
```

LINE 9 · STEP 7 OF 8

100 is not below 30, so the pump never starts. That's the symptom.

VARIABLES

```
raw = -1  
watering = false  
wet = 100
```

SERIAL MONITOR

```
[debug]  
raw=-1.00 wet=100.00
```

Find the bug with debug prints.

```
1 void loop() {  
2   float raw = incipe.getSoilMoisture();  
3   float wet = wetPercent(raw);  
4   if (raw = -1) wet = 100;           // meant: no reading → act as if wet  
5   Serial.print("[debug] raw=");  
6   Serial.print(raw);  
7   Serial.print(" wet=");  
8   Serial.println(wet);  
9   if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
10  if (watering && wet > STOP) { watering = false; Serial.println("PUMP OFF"); }  
11  delay(2000);  
12 }
```

LINE 4 · STEP 8 OF 8

The clue: the sensor said 85, the print says -1. Line 4 changed raw.

Fix: raw == -1.

VARIABLES

```
raw = -1  
watering = false  
wet = 100
```

SERIAL MONITOR

```
[debug]  
raw=-1.00 wet=100.00
```

The compiler may stay silent.

THE FIX

```
if (raw == -1) wet = 100;
```

With all warnings switched on, the compiler says “suggest parentheses around assignment used as truth value”.

With the default settings, the same code compiles without a word — so never count on a warning to catch it.

Fixed, a reading of 85 prints [debug] raw=85.00 wet=25.00, then PUMP ON.

Four habits.

Do	Why
Label every value: raw=, wet=	A bare 85 on the screen means nothing two minutes later.
Print before and after a suspect line	The bug is between the last right value and the first wrong one.
Print the state too: watering=1	Many bugs are “the program thinks it’s in a different state”.
Remove or switch them off when done	Extra prints slow the loop and hide real output.

A logic analyzer records signals over time.

Engineers use one to see two chips talking bit by bit, when a wired connection misbehaves.

For incipe.* code the board handles that conversation for you, so Serial prints are your first and best tool.

Smart Garden test plan.

#	Do this	Expected	Actual	Pass?
1	Probe in the air, with a debug print of wet	wet close to 0; PUMP ON once		
2	Then probe into a cup of water	wet close to 100; PUMP OFF once		
3	Damp soil between your thresholds	No new PUMP line — the state is kept		
4	Unplug the probe while the pump is off	Nothing switches on		
5	Unplug the probe while the pump is on	PUMP OFF on the next round		
6	Cover the light sensor (grow-light task)	GROW LIGHT ON once		

Game Console test plan.

#	Do this	Expected	Actual	Pass?
1	Leave the stick alone	No new frames		
2	Push right and hold for 5 s	One frame, you moved one pixel		
3	On pixel 0, push left	You stay on pixel 0		
4	Catch the fruit on pixel 4	score 1, the next fruit on pixel 1		
5	Unplug the joystick	Nothing moves — zone(-1) is 0		
6	Hold the push button 3 s, then let go (restart task)	One restart, not several		

Ask the AI to predict edge cases.

PASTE ONE PROJECT'S SKETCH INTO THE WORKSPACE AI CHAT WITH

Here is my Smart Garden sketch for an ESP32 board. List 6 edge cases it might meet (missing sensors, odd readings, timing). For each, predict exactly what my code prints, and suggest one robustness improvement in one sentence for a beginner. Do not rewrite the program, and do not change or add any incipe.* calls.

The AI can't run your board — it predicts. Your job is to check.

Check every prediction on the board.

Ask yourself	Why
Can I test each edge case on the board?	Add the testable ones as new rows in your test plan.
Did the prediction match what really printed?	Mark every row where the AI was wrong.
Does each suggestion keep the –1 fail-safe and the hysteresis?	“Robust” must never mean the pump runs when it shouldn’t.
Did it invent incipe.* functions or arguments?	Never accept that — compare with the Wiki.

Checkpoint 1

Buggy code, the sensor gives 85. What does the debug line print?

READ THE CODE

```
if (raw = -1) wet = 100;  
Serial.print("[debug] raw=");  
Serial.print(raw);  
Serial.print(" wet=");  
Serial.println(wet);
```

ANSWER

raw=-1.00 wet=100.00

WHY

Line 4 sets raw to -1 and wet to 100 before the print. (The line starts with [debug].)

**Fixed to ==, the sensor gives 85, watering is false.
What prints?**

READ THE CODE

```
if (raw == -1) wet = 100;  
...  
if (!watering && wet < START) ...
```

ANSWER

**raw=85.00 wet=25.00,
PUMP ON**

WHY

The debug line, then PUMP ON — 25 is below START (30).

Checkpoint 3

Fixed, the probe is unplugged (−1). What does the debug line print?

READ THE CODE

```
if (raw == -1) wet = 100;
```

ANSWER

raw=-1.00 wet=100.00

WHY

Exactly what the bug printed! A print alone can't tell "unplugged" from "bug" — plug the probe into dry soil and see whether raw changes.

Checkpoint 4

Fixed, watering is true, the probe is unplugged. What prints next?

READ THE CODE

```
if (watering && wet > STOP)
    ... Serial.println("PUMP OFF");
```

ANSWER

PUMP OFF

WHY

A -1 reading keeps wet at 100, above STOP (60). The fail-safe switches the pump off.

Checkpoint 5

**Game Console: you're on pixel 5 of 6 and push right.
What should the test plan say?**

READ THE CODE

```
player = constrain(player + z, 0, LEN - 1);
```

ANSWER

Stay on 5

WHY

constrain(6, 0, 5) is 5.

Three things to take away.

1

Debug like a detective: symptom, clue, hypothesis, change one thing, retest — and integrate one module at a time.

2

Labelled Serial prints show what the program believes; watch for = where you meant ==.

3

A test plan with expected results turns “it seems to work” into proof; the AI suggests edge cases, the board confirms them.

Next — M4 Session 1: Design thinking & problem statement. From building things right to building the right thing.